

Lab 04

OpenGL Basic Building Blocks: Other Primitives

Objectives

- Learning the remaining OpenGL primitives that can be used to specify lines, triangles and quads.

The methods to manage all the vertices and face indices will also be covered.

Special Instructions

READ these lab instructions BEFORE starting!

1. In Lab01, you were given 4 files – CGLabmain.cpp, CGLabmain.hpp, CGLab01.cpp and CGLab01.hpp files. We still need the first two files in this lab, but we will create the other NEW files on top of that later.
2. DO NOT copy-and-paste blindly! Some portions of code provided are just modifications, or additions of some lines of code. Always check carefully WHAT and WHERE to copy and paste. Most modifications/additions are in **BOLD**.

Programming Exercises

(80 minutes)

TASK 0: Revision regarding creating classes (10 minutes)

In this lab, we will create two new files (CGLab04.hpp and CGLab04.cpp) and add them into our project. Please refer to Lab02 on how to create and add new files to the project.

REMINDER: Before you do that, just as in all the previous labs, you will need to modify two lines in CGLabmain.cpp so that the lines,

```
...  
#include "CGLab01.hpp"  
  
using CGLab01::MyVirtualWorld;  
...
```

will be changed to...

```
...  
#include "CGLab04.hpp"  
  
using CGLab04::MyVirtualWorld;  
...
```

Please **REMEMBER** this. We may not remind you again in future labs.

The contents for CGLab04.hpp and CGLab04.cpp will be very similar to that of previous lab exercises, where we will create/define the class for a particular model in the .hpp file, and then put the implementation functions in the .cpp file. Remember that it is possible that we may have more than just one class. We will add more later.

Let's add a class called `MyUmbrella`. `CGLab04.hpp` will be modified (from previous `.hpp` files) as shown below:

```
/*
TCG6223 Computer Graphics
FIST, Multimedia University

CGLab04.hpp

Objective: Header File for Lab04 on Other Primitives
*/

#ifndef YP_CGLAB04_HPP
#define YP_CGLAB04_HPP

#include "CGLabmain.hpp"

namespace CGLab04 {

//class declaration
class MyUmbrella
{
public:
    void draw();
};

//-----
//the main program will call methods from this class
class MyVirtualWorld
{
public:

    //create an instance of the MyUmbrella class
    MyUmbrella myumbrella;

    void draw()
    {
        //call the draw() function of MyUmbrella class
        myumbrella.draw();
    }
    void tickTime()
    {
        //do nothing, reserved for doing animation
    }

    //for any one-time only initialization of the
    //  virtual world before any rendering takes place
    //  BUT after OpenGL has been initialized
    void init()
    {
    }
};
}; //end of namespace CGLab04

#endif //YP_CGLAB04_HPP
```

CGLab04.cpp will be as below:

```
/*
TCG6223 Computer Graphics
FIST, Multimedia University

CGLab04.cpp

Objective: Lab04 on Other Primitives
*/

#include <GL/glut.h>
#include "CGLab04.hpp"

using namespace CGLab04;

//class definition / implementation
void MyUmbrella::draw()
{
    //....
    //implementation code
    //....
}
```

Please take note that all the necessary steps to create one class, particularly the class declaration should go to the **CGLab04.hpp** header file while the class definition (i.e. implementation) should go to the **CGLab04.cpp** program file.

Note: You may have noticed that all the methods (member functions) of `MyVirtualWorld` class are defined (i.e. implemented) in the `CGLab04.hpp` header file. This is fine as long as those functions are extremely small, but generally it is advisable to put all your implementation code in the `CGLab04.cpp` program file.

From now on, whenever you need to create a class with certain drawing functions, you should know exactly what to do. Please try to REMEMBER this as we will not tell you again in future labs!!

Task 1: Using Other Primitives (30 minutes)

In all the previous labs, we have learned to use the `glBegin()` and `glEnd()` function as below:

```
glBegin( value );
    glColor3f(..., ..., ...);
    glVertex3f(..., ..., ...);
    ....
    glVertex3f(..., ..., ...);
    ....
    glColor3f(..., ..., ...);
    ....
    glVertex3f(..., ..., ...);
    ....
    glVertex3f(..., ..., ...);
glEnd();
```

The **value** in our previous examples can be either one of the values below:

GL_POINTS, GL_LINES, GL_TRIANGLES, GL_QUADS, GL_POLYGON

In actual fact, the **value** can also be any of the values below:

GL_LINE_STRIP, GL_LINE_LOOP, GL_QUAD_STRIP, GL_TRIANGLE_STRIP,
GL_TRIANGLE_FAN

Please refer to the OpenGL references on what these primitives are. We will explore some of these in this lab.

Refer back the code GL_TRIANGLE to draw the umbrella:**glDisable(GL_CULL_FACE);**

```
glBegin(GL_TRIANGLES);
    glColor3f(1.0f, 0.0f, 0.0f);
    glVertex3f( 0.0f, 5.0f, 0.0f ); //main center hub starting point
    glVertex3f(-5.0f, 0.0f, 5.0f ); //first fan face vertex
    glVertex3f( 0.0f, 0.0f, 7.0f ); //2nd

    glColor3f(1.0f, 1.0f, 0.0f );
    glVertex3f( 0.0f, 5.0f, 0.0f );
    glVertex3f( 0.0f, 0.0f, 7.0f );
    glVertex3f( 5.0f, 0.0f, 5.0f ); //third

    glColor3f(1.0f, 0.5f, 0.0f );
    glVertex3f( 0.0f, 5.0f, 0.0f );
    glVertex3f( 5.0f, 0.0f, 5.0f );
    glVertex3f( 7.0f, 0.0f, 0.0f ); //forth

    glColor3f(0.0f, 1.0f, 1.0f );
    glVertex3f( 0.0f, 5.0f, 0.0f );
    glVertex3f( 7.0f, 0.0f, 0.0f );
    glVertex3f( 5.0f, 0.0f, -5.0f ); //fifth
```

```
glColor3f(0.0f, 0.0f, 1.0f );
glVertex3f( 0.0f, 5.0f, 0.0f );
glVertex3f( 5.0f, 0.0f, -5.0f );
glVertex3f( 0.0f, 0.0f, -7.0f ); //sixth

glColor3f(1.0f, 0.0f, 1.0f );
glVertex3f( 0.0f, 5.0f, 0.0f );
glVertex3f( 0.0f, 0.0f, -7.0f );
glVertex3f(-5.0f, 0.0f, -5.0f ); //seventh

glColor3f(1.5f, 0.5f, 0.0f );
glVertex3f( 0.0f, 5.0f, 0.0f );
glVertex3f(-5.0f, 0.0f, -5.0f );
glVertex3f(-7.0f, 0.0f, 0.0f ); //eight

glColor3f(0.0f, 0.5f, 0.1f );
glVertex3f( 0.0f, 5.0f, 0.0f );
glVertex3f(-7.0f, 0.0f, 0.0f );
glVertex3f(-5.0f, 0.0f, 5.0f ); //and back to the first fan face vertex

//umbrella handle
glColor3f(0.6f, 0.5f, 0.1f );
glVertex3f( 0.0f, 5.0f, 0.0f );
glVertex3f(-1.0f, -10.0f, 1.0f );
glVertex3f( 1.0f, -10.0f, 1.0f );

glColor3f(0.3f, 0.8f, 0.4f );
glVertex3f( 0.0f, 5.0f, 0.0f );
glVertex3f( 1.0f, -10.0f, 1.0f );
glVertex3f(1.0f, -10.0f, -1.0f );

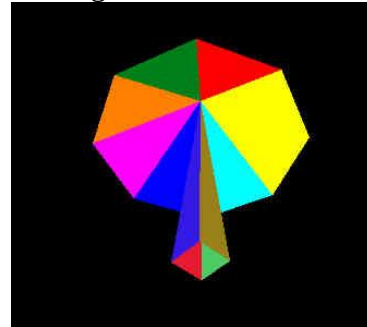
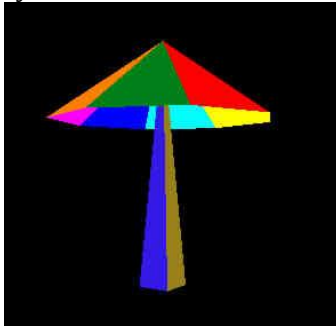
glColor3f(0.6f, 0.9f, 0.4f );
glVertex3f( 0.0f, 5.0f, 0.0f );
glVertex3f(1.0f, -10.0f, -1.0f );
glVertex3f(-1.0f, -10.0f, -1.0f );

glColor3f(0.9f, 0.1f, 0.2f );
glVertex3f( 0.0f, 5.0f, 0.0f );
glVertex3f(-1.0f, -10.0f, -1.0f );
glVertex3f(-1.0f, -10.0f, 1.0f );

glEnd();
glEnable(GL_CULL_FACE);
```

Task 1A: GL_TRIANGLE_FAN

In Lab02, you were asked to create the following object using the GL_TRIANGLE primitive:



Your task now is to create the same object using GL_TRIANGLE_FAN instead, which will be more efficient, as you will soon find out.

The code is as below:

```
void MyUmbrella::draw()
{
    glDisable(GL_CULL_FACE);

    glBegin(GL_TRIANGLE_FAN);
        //umbrella top
        glColor3f(1.0f, 0.0f, 0.0f );
        glVertex3f( 0.0f, 5.0f, 0.0f ); //main center hub starting point
        glColor3f(0.0f, 0.5f, 0.1f );
        glVertex3f(-5.0f, 0.0f, 5.0f ); //first fan face vertex
        glColor3f(1.0f, 0.5f, 0.5f );
        glVertex3f( 0.0f, 0.0f, 7.0f ); //2nd

        glColor3f(1.0f, 1.0f, 0.0f );
        glVertex3f( 5.0f, 0.0f, 5.0f ); //third

        glColor3f(1.0f, 1.0f, 0.0f );
        glVertex3f( 7.0f, 0.0f, 0.0f ); //forth

        glColor3f(0.0f, 1.0f, 1.0f );
        glVertex3f( 5.0f, 0.0f, -5.0f ); //fifth

        glColor3f(0.0f, 0.0f, 1.0f );
        glVertex3f( 0.0f, 0.0f, -7.0f ); //sixth

        glColor3f(1.0f, 0.0f, 1.0f );
        glVertex3f(-5.0f, 0.0f, -5.0f ); //seventh

        glColor3f(1.5f, 0.5f, 0.0f );
        glVertex3f(-7.0f, 0.0f, 0.0f ); //eight

        glColor3f(0.0f, 0.5f, 0.1f );
        glVertex3f(-5.0f, 0.0f, 5.0f ); //and back to the first fan face vertex

        //umbrella handle
        glColor3f(0.6f, 0.5f, 0.1f );
        glVertex3f( 0.0f, 5.0f, 0.0f );
        glColor3f(0.2f, 0.1f, 0.9f );
        glVertex3f(-1.0f, -10.0f, 1.0f );
        glColor3f(1.0f, 0.6f, 0.2f );
```

```

    glVertex3f( 1.0f, -10.0f,  1.0f );

    glColor3f(0.3f, 0.8f, 0.4f );
    glVertex3f( 1.0f, -10.0f, -1.0f );

    glColor3f(0.9f, 0.1f, 0.2f );
    glVertex3f(-1.0f, -10.0f, -1.0f );

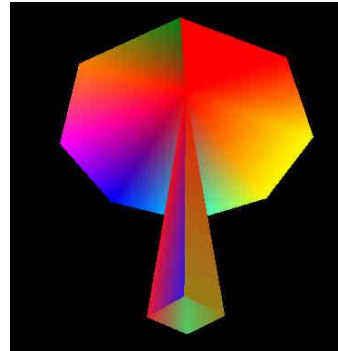
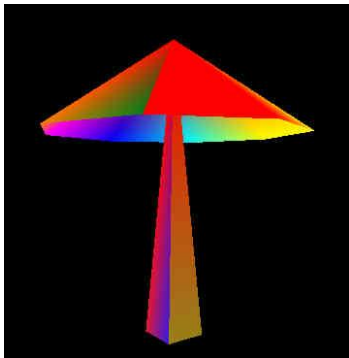
    glColor3f(0.2f, 0.1f, 0.9f );
    glVertex3f(-1.0f, -10.0f,  1.0f );

    glEnd();

    glEnable(GL_CULL_FACE);
}

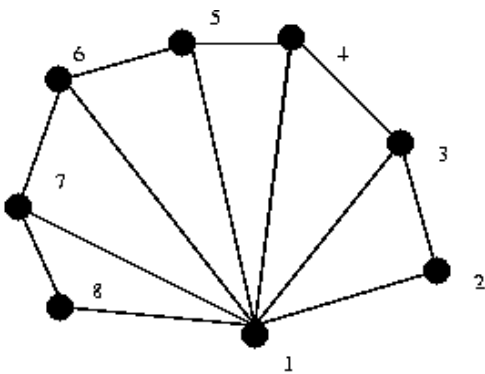
```

NOTE: The code above will not produce the same colour tone as the image given above, but instead you will get something like the following:



Understand the code and DISCUSS why we can never get the same colour as in Lab02?

first vertex of a triangle fan acts like a hub. The vertices following connect with the previous non-starting vertex and the hub.



Task 1B: GL_QUAD_STRIP

The following code is making use of the GL_QUAD_STRIP primitives:

```

void MyMMU::draw1()
{
    glDisable(GL_CULL_FACE);

    glBegin(GL_QUAD_STRIP);
        glColor3f(0.0f, 0.0f, 1.0f );
        glVertex3f(-15.0f, 0.0f, 0.0f );
        glVertex3f(-15.0f, 0.0f, 3.0f );
        glVertex3f(-15.0f, 8.0f, 0.0f );

```

```

    glVertex3f(-15.0f, 8.0f, 3.0f );
    glVertex3f(-10.0f, 5.0f, 0.0f );
    glVertex3f(-10.0f, 5.0f, 3.0f );
    glVertex3f( -5.0f, 8.0f, 0.0f );
    glVertex3f( -5.0f, 8.0f, 3.0f );
    glVertex3f( -5.0f, 0.0f, 0.0f );
    glVertex3f( -5.0f, 0.0f, 3.0f );
glEnd();
//solution
//draw another M
glBegin(GL_QUAD_STRIP);
glColor3f(0.0f, 1.0f, 0.0f );
glVertex3f(0.0f, 0.0f, 0.0f );
glVertex3f(0.0f, 0.0f, 3.0f );
glVertex3f(0.0f, 8.0f, 0.0f );
glVertex3f(0.0f, 8.0f, 3.0f );
glVertex3f(5.0f, 5.0f, 0.0f );
glVertex3f(5.0f, 5.0f, 3.0f );
glVertex3f( 10.0f, 8.0f, 0.0f );
glVertex3f( 10.0f, 8.0f, 3.0f );
glVertex3f( 10.0f, 0.0f, 0.0f );
glVertex3f( 10.0f, 0.0f, 3.0f );
glEnd();
//draw a U

glBegin(GL_QUAD_STRIP);
glColor3f(1.0f, 0.0f, 0.0f );
glVertex3f( 15.0f, 8.0f, 0.0f );
glVertex3f( 15.0f, 8.0f, 3.0f );
glVertex3f( 15.0f, 4.0f, 0.0f );
glVertex3f( 15.0f, 4.0f, 3.0f );
glVertex3f( 17.0f, 0.0f, 0.0f );
glVertex3f( 17.0f, 0.0f, 3.0f );
glVertex3f( 20.0f, 0.0f, 0.0f );
glVertex3f( 20.0f, 0.0f, 3.0f );
glVertex3f( 22.0f, 4.0f, 0.0f );
glVertex3f( 22.0f, 4.0f, 3.0f );
glVertex3f( 22.0f, 8.0f, 0.0f );
glVertex3f( 22.0f, 8.0f, 3.0f );
glEnd();
//end of solution
glEnable(GL_CULL_FACE);
}

```

GL_QUAD_STRIP: Draws a connected group of quadrilaterals

The expected output of the implementation above would be as shown in Figure 1.



Figure 1



Figure 2

Your task now is to add and modify the code in order to draw the letters “M M U” (Figure 2).

Task 2 : Using Vertices List (20 minutes)

The example in [Task 1B](#) can be improved using the code below:

```
void MyMMU::draw2()
{
    glDisable(GL_CULL_FACE);

    static GLfloat vertices[][3] =
    { {-15.0f, 0.0f, 0.0f }, // vertex 0
      {-15.0f, 0.0f, 3.0f }, // vertex 1
      {-15.0f, 8.0f, 0.0f }, // vertex 2
      {-15.0f, 8.0f, 3.0f }, // vertex 3
      {-10.0f, 5.0f, 0.0f }, // vertex 4
      {-10.0f, 5.0f, 3.0f }, // vertex 5
      { -5.0f, 8.0f, 0.0f }, // vertex 6
      { -5.0f, 8.0f, 3.0f }, // vertex 7
      { -5.0f, 0.0f, 0.0f }, // vertex 8
      { -5.0f, 0.0f, 3.0f }  // vertex 9
    };

    glBegin(GL_QUAD_STRIP);
    glColor3f(0.0f, 0.0f, 1.0f );
    glVertex3fv( vertices[0] );
    glVertex3fv( vertices[1] );
    glVertex3fv( vertices[2] );
    glVertex3fv( vertices[3] );
    glVertex3fv( vertices[4] );
    glVertex3fv( vertices[5] );
    glVertex3fv( vertices[6] );
    glVertex3fv( vertices[7] );
    glVertex3fv( vertices[8] );
    glVertex3fv( vertices[9] );
    glEnd();

    glEnable(GL_CULL_FACE);
}
```

We replace the `glVertex3f( ..., ..., ... )` by `glVertex3fv( array )` where the ‘v’ stands for ‘vector’ (or array) of numbers. ‘3f’ simply means we want to use 3 floating-point numbers (for each dimension) from the array.

The program can still be improved further using the code below:

```
void MyMMU::draw3()
{
    glDisable(GL_CULL_FACE);

    const int numVertices = 10;

    static GLfloat vertices[][3] =
    { {-15.0f, 0.0f, 0.0f }, // vertex 0
      {-15.0f, 0.0f, 3.0f }, // vertex 1
      {-15.0f, 8.0f, 0.0f }, // vertex 2
      {-15.0f, 8.0f, 3.0f }, // vertex 3
      {-10.0f, 5.0f, 0.0f }, // vertex 4
      {-10.0f, 5.0f, 3.0f }, // vertex 5
    }
```

```

        { -5.0f, 8.0f, 0.0f }, // vertex 6
        { -5.0f, 8.0f, 3.0f }, // vertex 7
        { -5.0f, 0.0f, 0.0f }, // vertex 8
        { -5.0f, 0.0f, 3.0f } // vertex 9
    };

    glBegin(GL_QUAD_STRIP);
    glColor3f(0.0f, 0.0f, 1.0f );
    for (int i=0; i<numOfVertices; ++i)
    {
        glVertex3fv ( vertices[i] );
    }
    glEnd();

    glEnable(GL_CULL_FACE);
}

```

Task 3 : Using Vertex Lists and Face Lists (30 minutes)

Task 3A (10 minutes)

If you have noticed all the previous examples, whenever a vertex forms part of more than one polygon, that vertex would be repeated many times. This makes your code longer and harder to maintain and the execution time would also take longer. The solution for that is to have a **Vertex List** as well as **Face List**, as shown in the example below:

```

void MyBox::draw()
{
    static GLfloat vertices[][3] =
    { { -5.0f, -5.0f, 5.0f }, // vertex 0
      { 5.0f, -5.0f, 5.0f }, // vertex 1
      { 5.0f, -5.0f, -5.0f }, // vertex 2
      { -5.0f, -5.0f, -5.0f }, // vertex 3
      { -5.0f, 5.0f, 5.0f }, // vertex 4
      { 5.0f, 5.0f, 5.0f }, // vertex 5
      { 5.0f, 5.0f, -5.0f }, // vertex 6
      { -5.0f, 5.0f, -5.0f } // vertex 7
    };

    //vertex indices of each polygon
    static int faces[] =
    {
        3, 0, 4, 7, //face 0
        3, 2, 1, 0, //face 1
        2, 3, 7, 6, //face 2
        1, 2, 6, 5, //face 3
        4, 5, 6, 7, //face 4
        0, 1, 5, 4 //face 5
    };

    static GLfloat colors[][3] =
    {
        { 0.0f, 0.0f, 1.0f },
        { 0.0f, 1.0f, 1.0f },
        { 0.0f, 1.0f, 0.0f },
        { 1.0f, 1.0f, 0.0f },
        { 1.0f, 0.0f, 0.0f },
        { 1.0f, 0.0f, 1.0f }
    }
}

```

```

};

int numberOfFaces = 6;
int verticesPerFace = 4;
int n=0;

for (int j=0; j<numberOfFaces; j++)
{
    glColor3fv( colors[j] );
    glBegin(GL_POLYGON);
        for (int k=0; k<verticesPerFace; ++k)
        {
            glVertex3fv(vertices[ faces[n] ]);
            ++n;
        }
    glEnd();
}
}

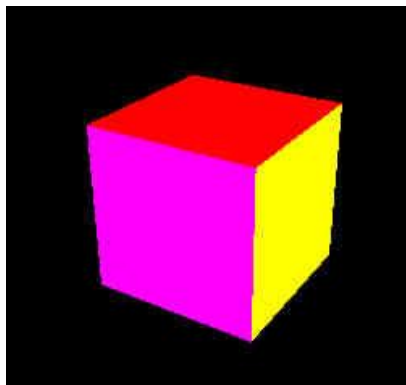
```

The Vertex List stores a list of vertices with their coordinates (x, y, and z).

The Face List stores a list of faces of an object, with their respective vertex indices.

In the code above, the colors are also stored as a list (maybe you can call it a **Color List!**). Why? Exactly the same reason!

If you run this, you will see the following output:

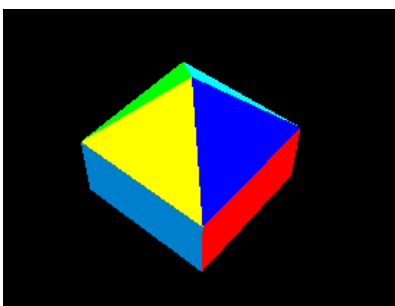


Take note that all the indices start at zero (first element of an array is zero index).

Understand the code and modify it to draw other shapes.

REMEMBER: The sequence of your vertices for each polygon face must be COUNTER-CLOCKWISE.

Try to use the face list concept, modify the coordinate to draw the following figure.



Task 3B (20 minutes)

The example above has a limitation that each polygon face can only have 4 vertices. Modify the code such that you can have any number of vertices for each polygon. How would you do that?

TIP: You may have one integer array to keep track of how many vertices are there in each polygon. However, the better way is to add one addition number (normally called a “sentinel number”) to the (end of the) face list to indicate the end of the vertices list for that particular polygon. To do that, part of the code has been given to you below. **You have to complete it to make it work.**

```
void MyDiamond::draw()
{
    static GLfloat vertices[][3] =
    {
        //top tip
        { 0.0f, 10.0f, 0.0f},
        //central upper
        { 10.0f, 2.0f, 0.0f},
        { 5.0f, 2.0f, 8.5f},
        { -5.0f, 2.0f, 8.5f},
        { -10.0f, 2.0f, 0.0f},
        { -5.0f, 2.0f, -8.5f},
        { 5.0f, 2.0f, -8.5f},
        //central lower
        { 10.0f, -2.0f, 0.0f},
        { 5.0f, -2.0f, 8.5f},
        { -5.0f, -2.0f, 8.5f},
        { -10.0f, -2.0f, 0.0f},
        { -5.0f, -2.0f, -8.5f},
        { 5.0f, -2.0f, -8.5f},
        //bottom tip
        { 0.0f, -10.0f, 0.0f}
    };

    //vertex indices of each polygon
    //-1 means end of polygon,
    //-999 means end of faces list
    static int faces[] =
    {
        0, 1, 6, -1,
        0, 6, 5, -1,
        0, 5, 4, -1,
        0, 4, 3, -1,
        0, 3, 2, -1,
        0, 2, 1, -1,

        1, 7, 12, 6, -1,
        6, 12, 11, 5, -1,
        5, 11, 10, 4, -1,
        4, 10, 9, 3, -1,
        3, 9, 8, 2, -1,
        2, 8, 7, 1, -1,

        7, 8, 9, 10, 11, 12, -1,

        -999 //end
    };

    static GLfloat colors[][3] =
    { {0.0f, 0.0f, 1.0f },
```

```

    {0.0f, 1.0f, 1.0f },
    {0.0f, 1.0f, 0.0f },
    {1.0f, 1.0f, 0.0f },
    {1.0f, 0.0f, 0.0f },
    {1.0f, 0.0f, 1.0f },
    {0.0f, 0.5f, 1.0f },
    {0.0f, 1.0f, 0.5f },
    {0.5f, 0.5f, 0.0f },
    {1.0f, 0.0f, 0.5f },
    {0.5f, 0.0f, 1.0f },
    {0.0f, 0.5f, 0.5f },
    {0.5f, 1.0f, 0.0f }
};

int n=0; //n iterate through each numbers in 'faces' array
int facecount=0;

while (faces[n] != -999)
{
    glColor3fv( colors[ facecount ] );
    glBegin(GL_POLYGON);

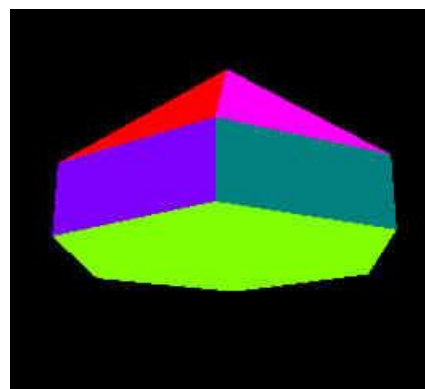
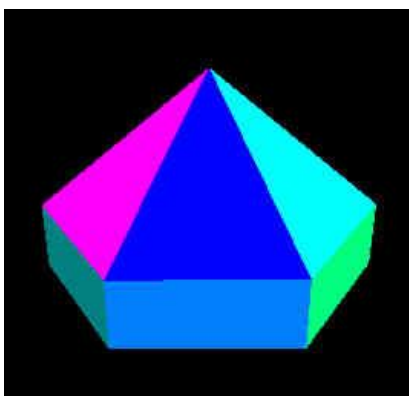
    // fill this up - solution
    while (faces[n] != -1)
    {
        // fill this up - solution

    }

    glEnd();
    ++n;
    ++facecount;
}
}

```

Complete the program so that you get this output as viewed from two different views:



HAVE FUN!

Extra Exercises

VERY IMPORTANT

BEFORE coming to the next lab, you MUST:

- 1. COMPLETE all the exercises in this lab!**
- 2. DO the extra exercises below (if there are any)!**
- 3. READ the lab instructions!**
- 4. PREPARE files for next lab!**

Please try and attempt **Task 1A, 1B, 1C and 1D** of the next lab, **Lab05** on your own before the next lab session.

1. In real life, a model could have hundreds of thousands of vertices and polygons, thus very often information such as vertices coordinate, colours information, faces indices etc are saved in files and read into the program. One example is the code that draws the dragon model in Lab01. Check out the program and see how this is done.

Convert all your programs so that these vertices and polygons information are saved in files and your program will read them into the array when executed.

2. There are many free models that you can use to populate your virtual world. A very famous site is at Stanford University. The dragon model was obtained from this site as well. The URL is:

<http://graphics.stanford.edu/data/3Dscanrep/>

* END OF LAB *